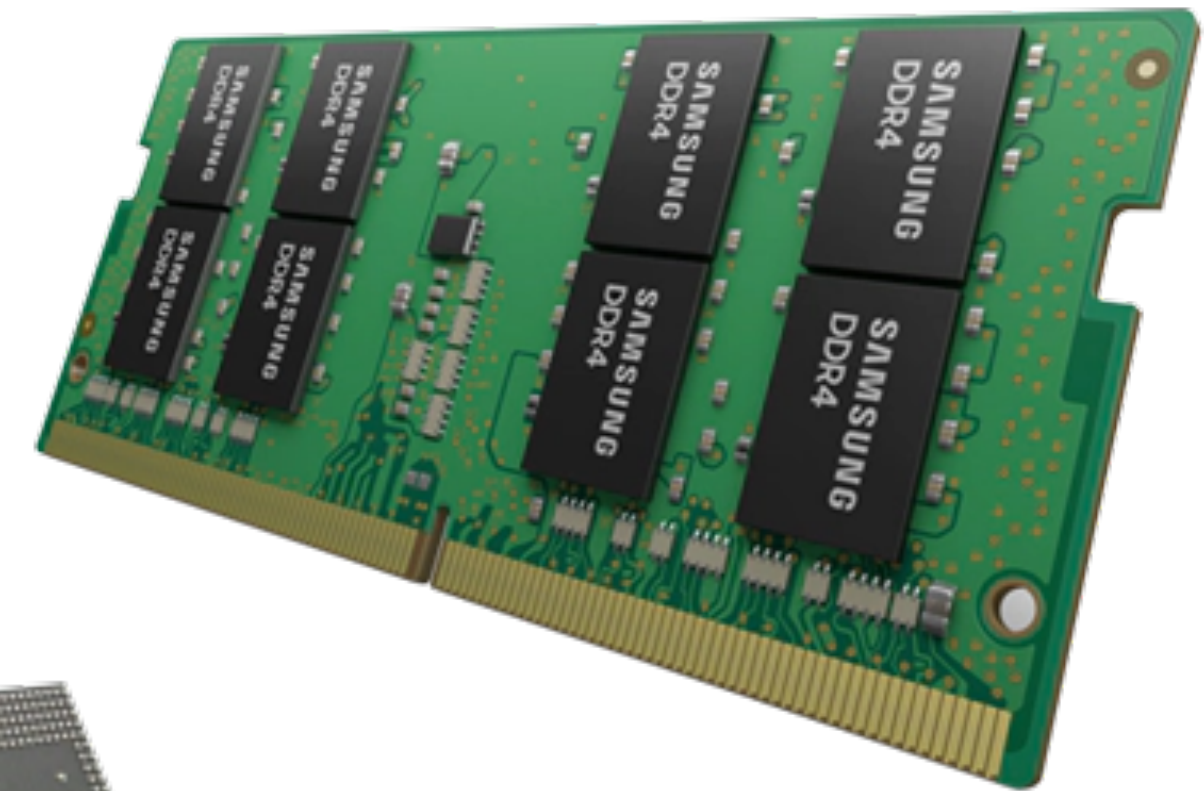


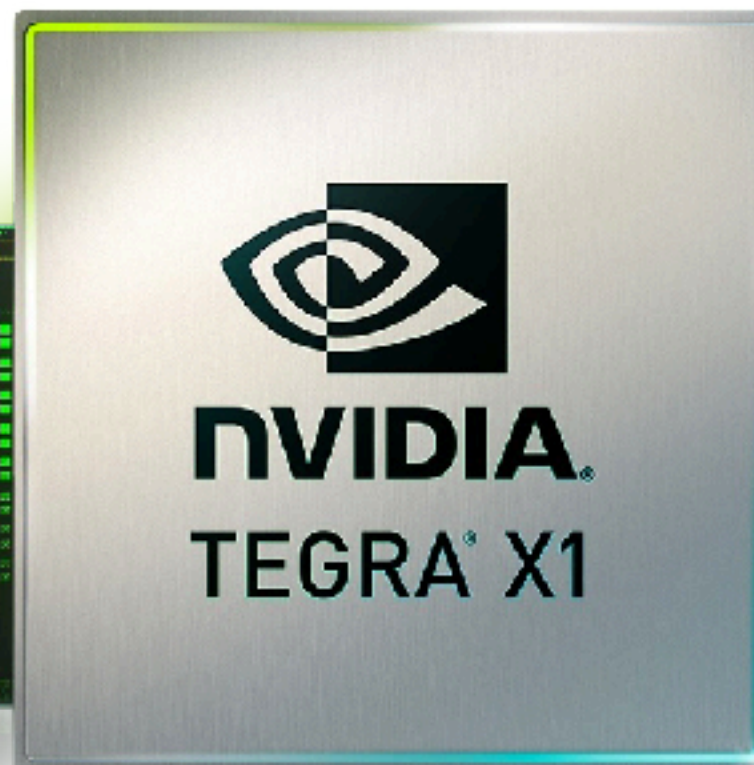
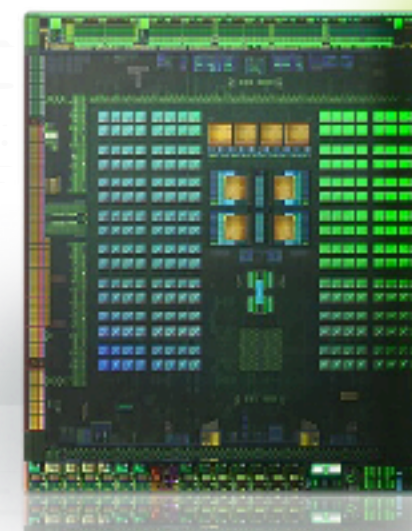
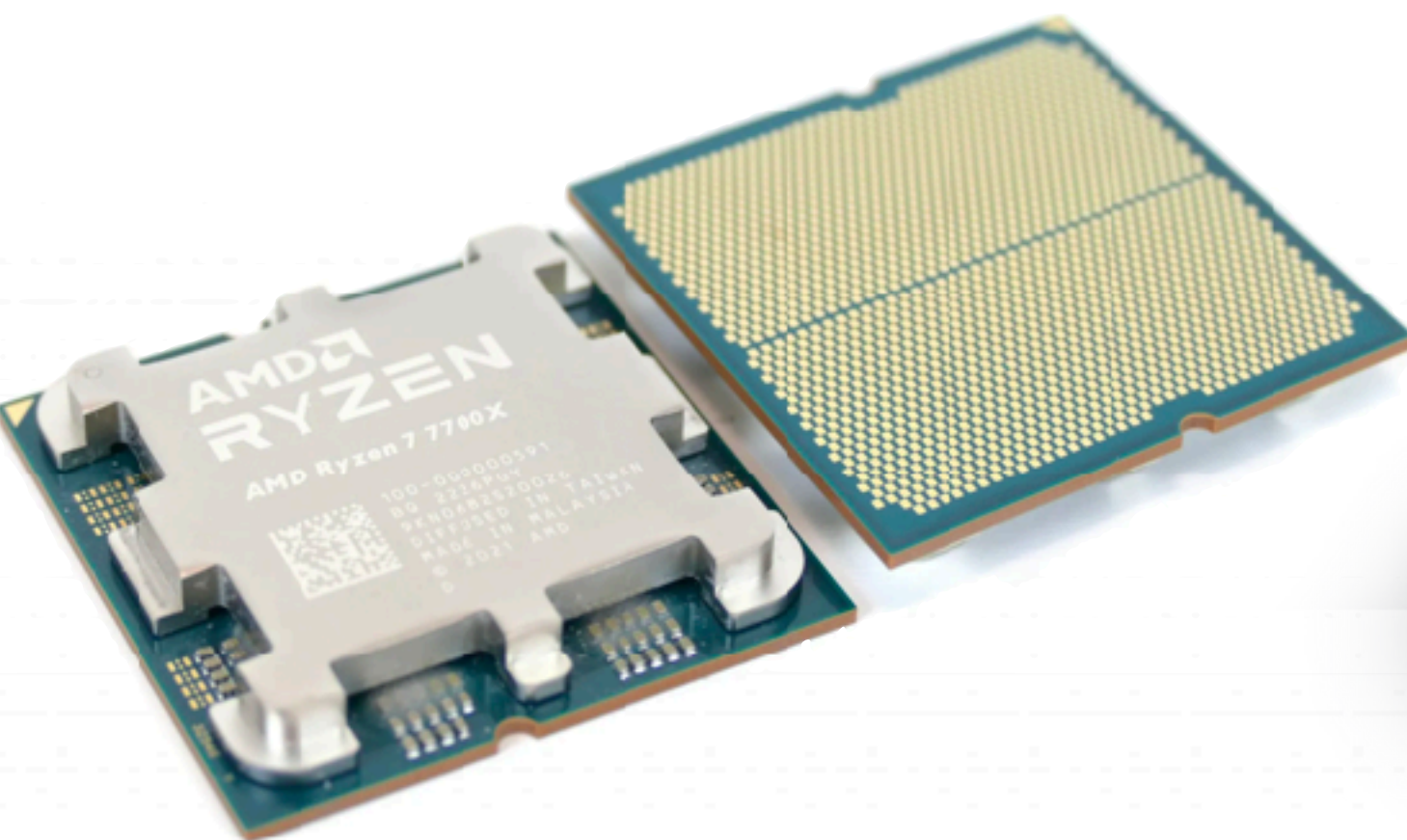
Performance (1): What does "perfect" mean?

Hung-Wei Tseng

Recap: Processors and memory modules are everywhere!



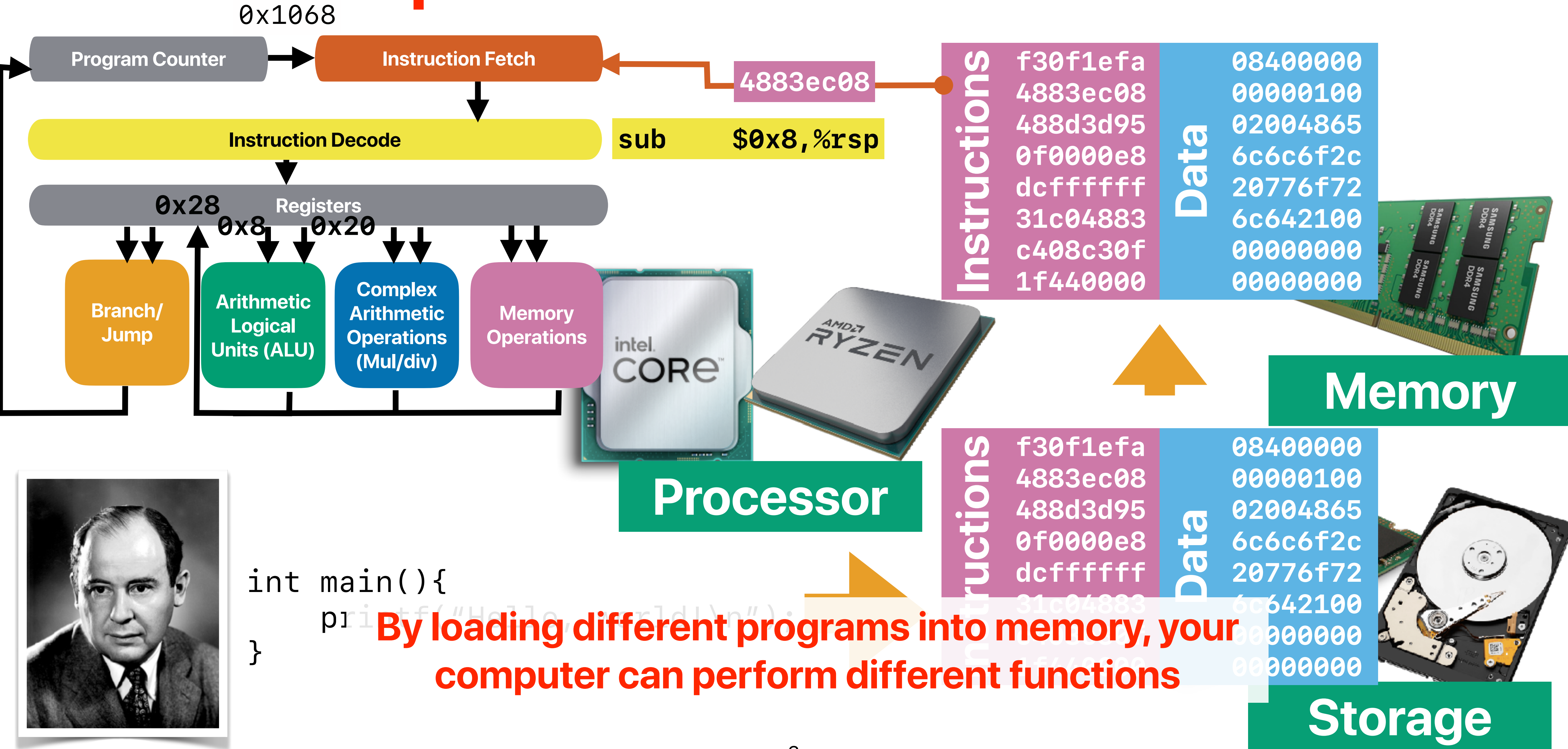
Processors



Memory



Recap: von Neumann architecture



Recap: Demo

```
if(option)
    std::sort(data, data + arraySize);
for (unsigned c = 0; c < arraySize*1000; ++c) {
    int t = std::rand();

    (data[c%arraySize] >= t)
        sum ++;
}
```

$O(n \log_2 n)$

$O(n)$

if option is set to 1: $O(n \log_2 n)$ — but faster!!!

otherwise, O(n): $O(n)$

Recap: Demo (2)

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

$O(n^2)$

A Lot Better!

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

Complexity

$O(n^2)$

Performance?

Worse

**Do you think UCR is a good
school?**

Best Nation

Schools in the National Univer
a full range of undergraduate r
producing groundbreaking res

To unlock full rankings, SAT/A

SUMMARY ▾



Best Computer

Best Computer Science Schools

Ranked in 2022, part of [Best Science S](#)

Earning a graduate degree in computer
technology companies and colleges ar
reflects its average rating on a scale fr
institutions. [Read the methodology »](#)



UC San Diego Ranked No. 1 Public University by Washington Monthly

Campus celebrated as a leader in social mobility, research and public

443

Scho

Sc

Loca

Cit

All

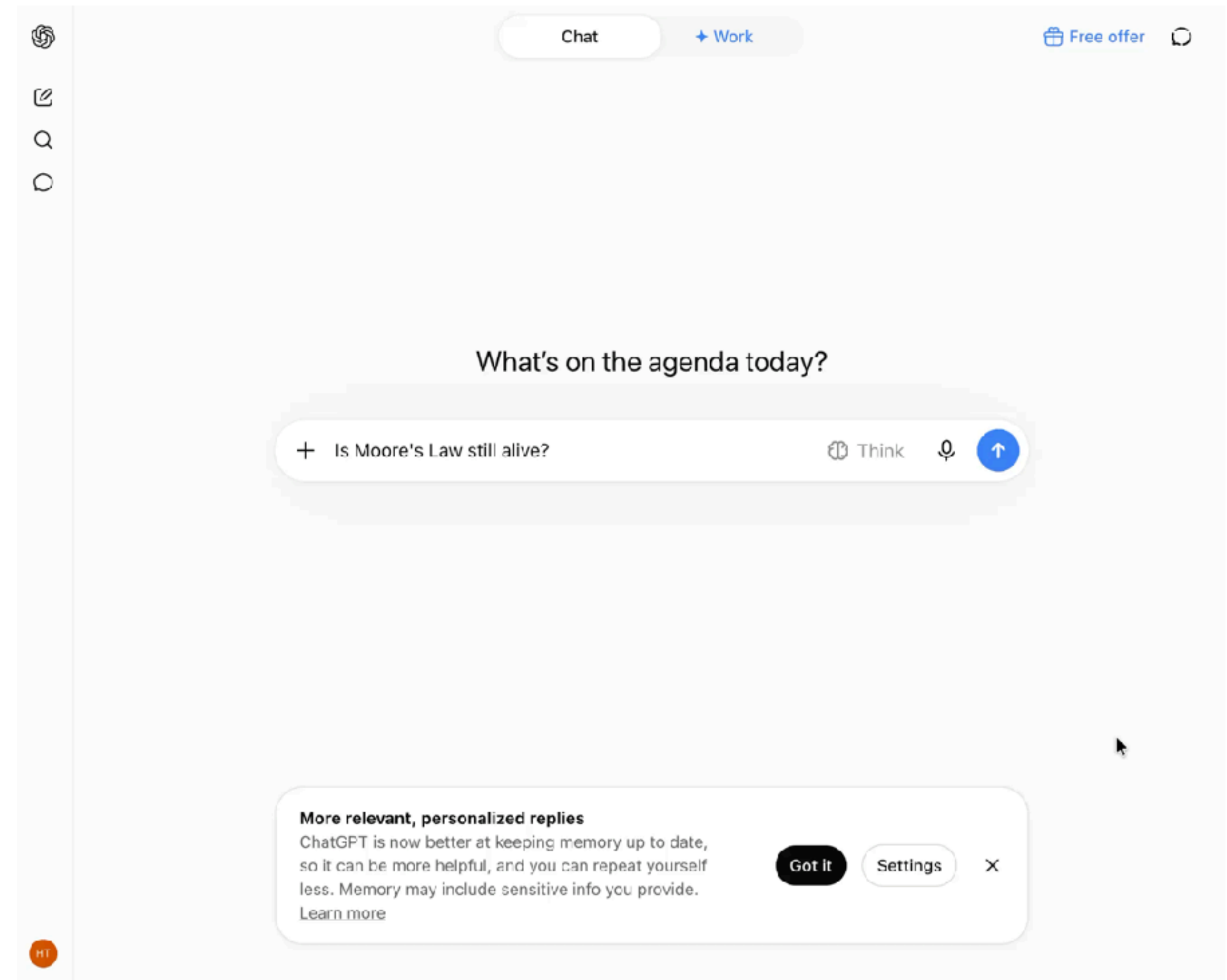
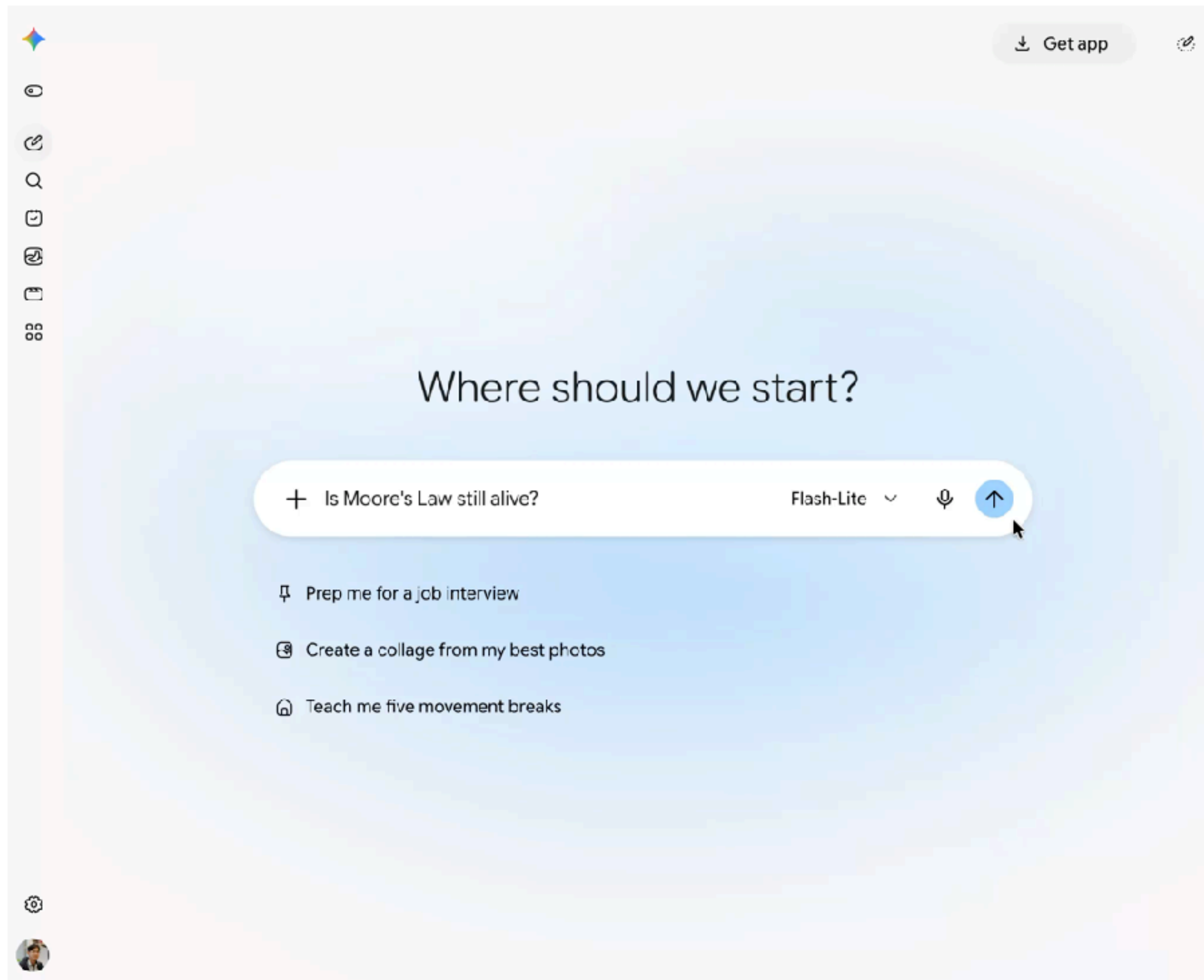
Ranki

Natio

U.S. News names UCR No. 1 public for social mobility

UC Riverside has been the top public university in the nation every year since the social mobility category's inception

Moore's law — which side's performance is better?



What do you compare matters!

Outline

- Definition of "Performance"
- The classical CPU performance equation
- What about metrics on throughputs? (e.g., FLOPS)
- What affects performance
 - Programming languages
 - Programmers

Performance equation

- Consider the following c code snippet and x86 instructions implement the code snippet
If (1) `data_size` is set to 1,000,000,000,
(2) a memory instruction takes 5 cycles,
(3) a branch/jump instruction takes 2 cycles,
(4) other instructions takes 1 cycle on average,
and (5) the processor runs at 4 GHz,
how much time is it take to finish executing the code snippet?

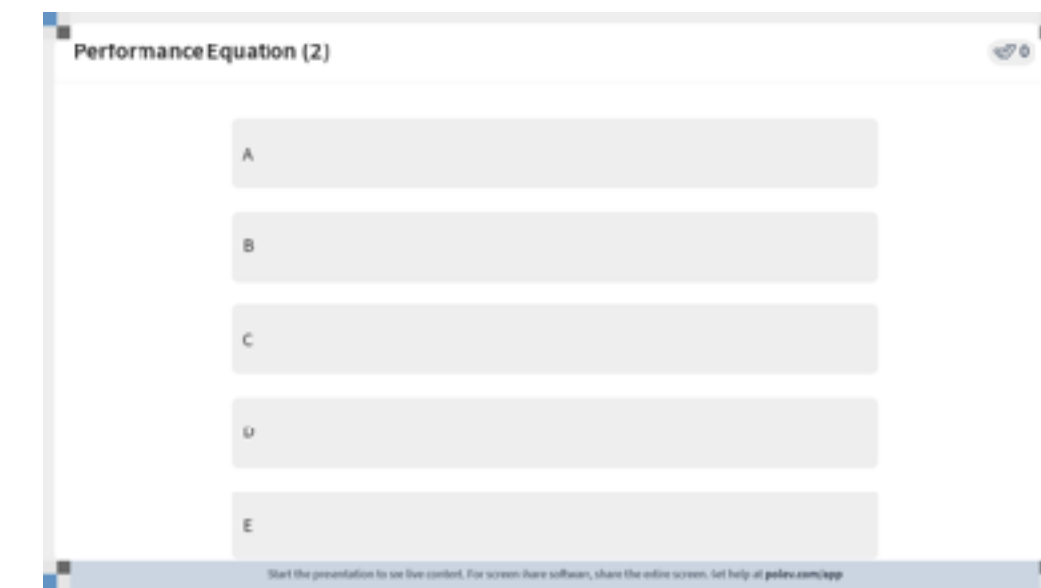
- A. 0.5 sec
- B. 1 sec
- C. 2.5 sec
- D. 3.75 sec
- E. 4 sec

```
int init_data(int64_t *data, int
data_size) {
    register unsigned int i = 0;
    for(i = 0; i < data_size; i++) {
        s+=data[i];
    }
    return s;
}
```

C

x86

```
.LFB16:
    endbr64
    testl    %esi, %esi
    jle     .L2
    leal     -1(%rsi), %ecx
    xorq     %rax, %rax
.L3:
    movslq   (%rdi), %rdx
    addq     $4, %rdi
    addq     %rdx, %rax
    cmpq     %rcx, %rdi
    jne     .L3
.L2:
    xorlq    %rax, %rax
    ret
```



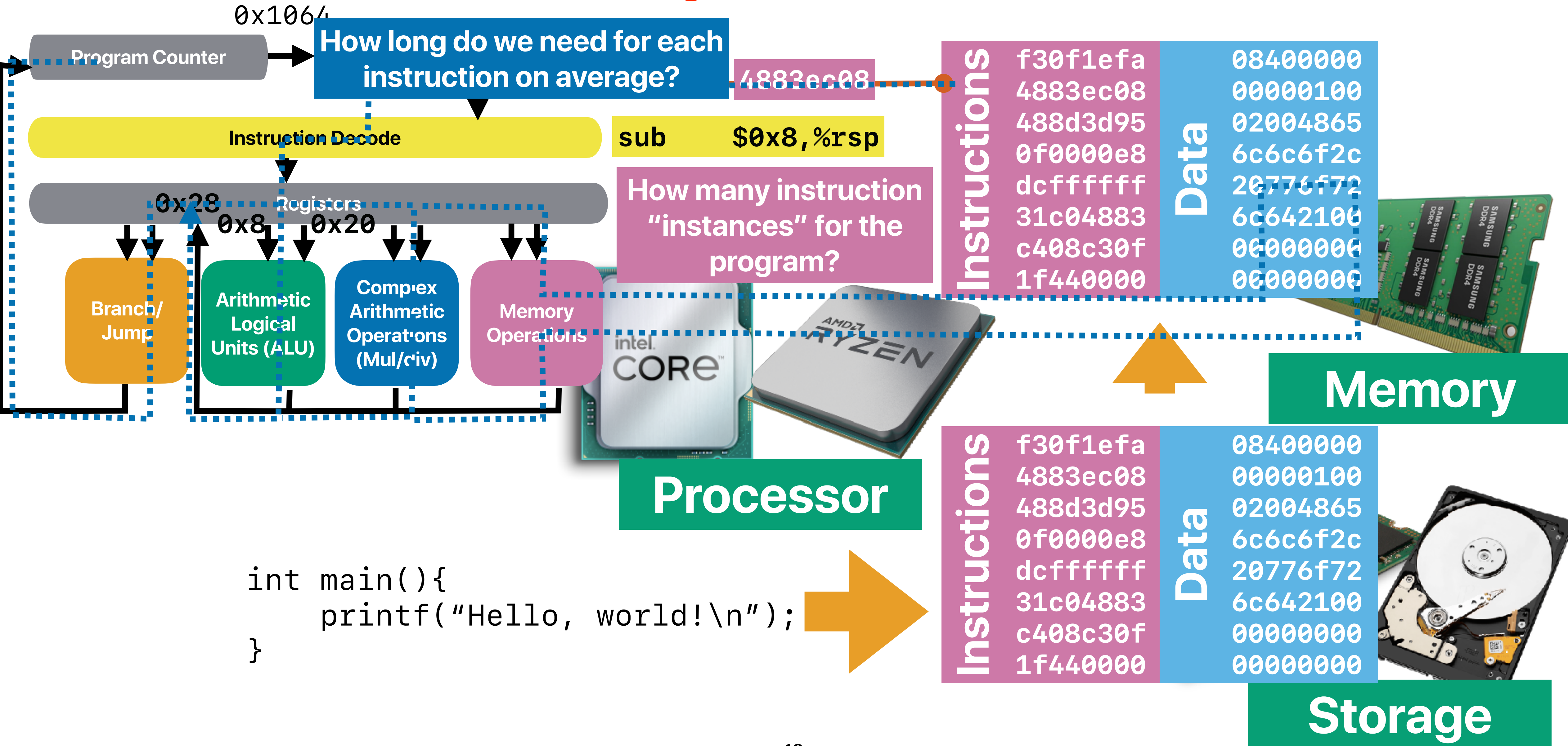
Performance equation

- Consider the following c code snippet and x86 instructions implement the code snippet
If (1) `data_size` is set to 1,000,000,000,
(2) a memory instruction takes 5 cycles,
(3) a branch/jump instruction takes 2 cycles,
(4) other instructions takes 1 cycle on average,
and (5) the processor runs at 4 GHz,
how much time is it take to finish executing the code snippet?

- A. 0.5 sec
- B. 1 sec
- C. 2.5 sec
- D. 3.75 sec
- E. 4 sec

C	x86
<pre>int init_data(int64_t *data, int data_size) { register unsigned int i = 0; for(i = 0; i < data_size; i++) { s+=data[i]; } return s; }</pre>	<pre>.LFB16: endbr64 testl %esi, %esi jle .L2 leal -1(%rsi), %ecx xorq %rax, %rax .L3: movslq (%rdi), %rdx addq \$4, %rdi addq %rdx, %rax cmpq %rcx, %rdi jne .L3 .L2: xorlq %rax, %rax ret</pre>

Execution time of a program in the von Neumann model



Definition of performance in Computer Architecture

$$\textit{Performance} = \frac{1}{\textit{Execution Time}}$$

Classic CPU Performance Equation (ET of a program)

How many instruction
"instances" for the
program?

×

How long do we need for each
instruction on average?

$$Execution\ Time = \frac{Instructions}{Program} \times \frac{Cycles}{Instruction} \times \frac{Seconds}{Cycle}$$

C Code

```
int init_data(int64_t *data, int data_size) {
    register unsigned int i = 0;
    for(i = 0; i < data_size; i++) {
        s+=data[i];
    }
    return s;
}

int main(int argc, char **argv) {
    int *data = malloc(8000000000);
    init_data(data, 1000000000)
    return 0;
}
```

1000000000x

x86 instructions

```
init_data:
.LFB16:
    endbr64
    testl    %esi, %esi
    jle     .L2
    leal     -1(%rsi), %ecx
    xorq     %rax, %rax
.L3:
    movslq   (%rdi), %rdx
    addq     $4, %rdi
    addq     %rdx, %rax
    cmpq     %rcx, %rdi
    jne     .L3
.L2:
    xorlq    %rax, %rax
    ret
```

If data memory access instructions takes 5 cycles, branch 2 cycles, others take only 1 cycle, CPU freq. = 4 GHz

$$CPI_{average} = 20\% \times 5 + 20\% \times 2 + 60\% \times 1 = 2$$

memory inst.

$$ET = (5 \times 10^9) \times 2 \times \frac{1}{4 \times 10^9} \text{ sec} = 2.5 \text{ sec}$$

←branch inst.

Performance equation

- Consider the following c code snippet and x86 instructions implement the code snippet

C	x86
<pre>for(i = 0; i < count; i++) { s += a[i]; }</pre>	<pre>.L3: movslq (%rdi), %rdx addq \$4, %rdi addq %rdx, %tax cmpq %rcx, %rdi jne .L3</pre>

If (1) count is set to 1,000,000,000, (2) a memory instruction takes 4 cycles, (3) a branch/jump instruction takes 3 cycles, (4) other instructions takes 1 cycle on average, and (5) the processor runs at 4 GHz, how much time is it take to finish executing the code snippet?

- A. 0.5 sec
- B. 1 sec
- C. 2.5 sec
- D. 3.75 sec
- E. 4 sec

$$ET = IC \times CPI \times CT$$

$$ET = (5 \times 10^9) \times (20\% \times 5 + 20\% \times 2 + 60\% \times 1) \times \frac{1}{4 \times 10^9} \text{ sec} = 2.5 \text{ sec}$$

**total # of dynamic
instructions**

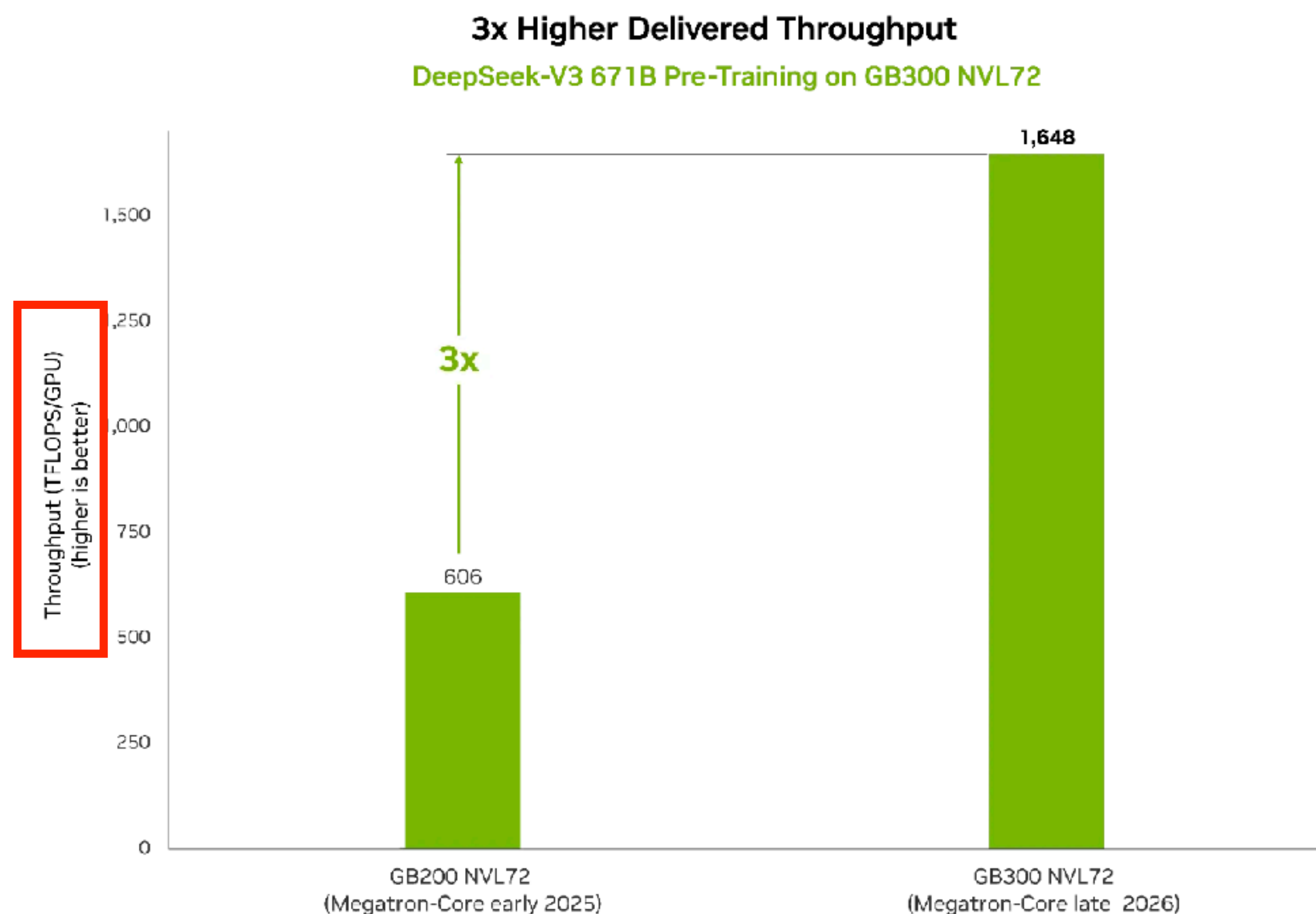
average CPI

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric —
Instruction count, cycles per instruction, cycle time define the
latency of execution on CPUs

What about FLOPS?

On the DeepSeek-V3 671B model, using 256 GPUs, Megatron Core reaches **1,648 TFLOPs per GPU** on GB300 NVL72, compared to **606 TFLOPs per GPU** on earlier GB200 NVL72 results, ~3x higher delivered throughput per GPU in a single generation.



25
 Figure 3. 3x higher delivered performance GB200 NVL72 (early software version) vs GB300 NVL72 (latest software version)

TFLOPS (Tera FLoating-point Operations Per Second)

$$TFLOPS = \frac{\# \text{ of floating point instructions} \times 10^{-12}}{\text{Exection Time}}$$

Let's measure the TFLOPS of matrix multiplications

```
for(i = 0; i < M; i++) {  
  for(j = 0; j < N; j++) {  
    for(k = 0; k < K; k++) {  
      c[i][j] += a[i][k]*b[k][j];  
    }  
  }  
}
```

Floating point operations per second (FLOP"S"):

Floating point operations (FLOP"s"):

$$M \times N \times K \times 2$$

Given $M = N = K = 2048$

$$2^{3 \times 11} \times 2 = 2^{34} \quad \textbf{FLOPs in total}$$

$$TFLOPS = \frac{2^{34}}{ET_{seconds} \times 2^{12}}$$



How reflective is FLOPS?

- If we're given the FLOPS of the underlying GPU of the machine, how many situations below can the FLOPS be representative to the real performance?
 - ① The FLOPS remains the same if we change the dataset for the same program
 - ② The FLOPS remains the same if we change the data type to double
 - ③ The FLOPS remains the same if we run a different the algorithm
 - ④ The FLOPS ratio of two computers running the same program will be the speedup of running the same program on these two computers
- A. 0
B. 1
C. 2
D. 3
E. 4

A screenshot of a Pollev poll interface. It shows five empty input boxes labeled A, B, C, D, and E, arranged vertically. The interface is clean and modern, with a light gray background and a blue header bar.

How reflective is FLOPS?

- If we're given the FLOPS of the underlying GPU of the machine, how many situations below can the FLOPS be representative to the real performance?
 - ① The FLOPS remains the same if we change the dataset for the same program
 - ② The FLOPS remains the same if we change the data type to double
 - ③ The FLOPS remains the same if we run a different the algorithm
 - ④ The FLOPS ratio of two computers running the same program will be the speedup of running the same program on these two computers
- A. 0
B. 1
C. 2
D. 3
E. 4

Demo: matmul on GPU

Size	Latency	Relative Latency	Throughput (Output Numbers Per Second)	Relative Throughput
16x16x16	~ 0.09ms	1	0.09ms/256	1
32x32x32	~ 0.09ms	1	0.09ms/1024	4
64x64x64	~ 0.09ms	1	0.09ms/4096	16

Larger throughput doesn't mean shorter latency!

Is TFLOPS (Tera Floating-point Operations Per Second) a good metric?

$$\begin{aligned} TFLOPS &= \frac{\# \text{ of floating point instructions} \times 10^{-12}}{\text{Execution Time}} \\ &= \frac{IC \times \% \text{ of floating point instructions} \times 10^{-12}}{IC \times CPI \times CT} \\ &= \frac{\% \text{ of floating point instructions} \times 10^{-12}}{CPI \times CT} \end{aligned} \quad \text{IC is gone!}$$

- If we have more iterations? Larger datasets? — potentially changes the IC
- What if the hardware trade (cheat) performance with accuracy?
- Cannot compare different ISA/compiler
 - What if the compiler can generate code with fewer instructions?
 - What if new architecture has more IC but also lower CPI?
- If floating point operations are not critical in the target application?

How reflective is FLOPS?

- If we're given the FLOPS of the underlying GPU of the machine, how many situations below can the FLOPS be representative to the real performance?
 - ① The FLOPS remains the same if we change the dataset for the same program — IC changed
 - ② The FLOPS remains the same if we change the data type to double — CPI changed
 - ③ The FLOPS remains the same if we run a different the algorithm — IC changed
 - ④ The FLOPS ratio of two computers running the same program will be the speedup of running the same program on these two computers — IC/CPI/CT changed
- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

Latency v.s. Bandwidth/Throughput

- Latency — the amount of time to finish an operation
 - End-to-end execution time of "something"
 - Access time
 - Response time
- Throughput — the amount of work can be done within a given period of time (typically "something" per "timeframe" or the other way around)
 - Bandwidth (MB/Sec, GB/Sec, Mbps, Gbps)
 - IOPs (I/O operations per second)
 - FLOPs (Floating-point operations per second)
 - IPS (Inferences per second)
 - FPS (Frames per second)

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric —
Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions

What Affects Each Factor in Performance Equation

Programming languages & performance



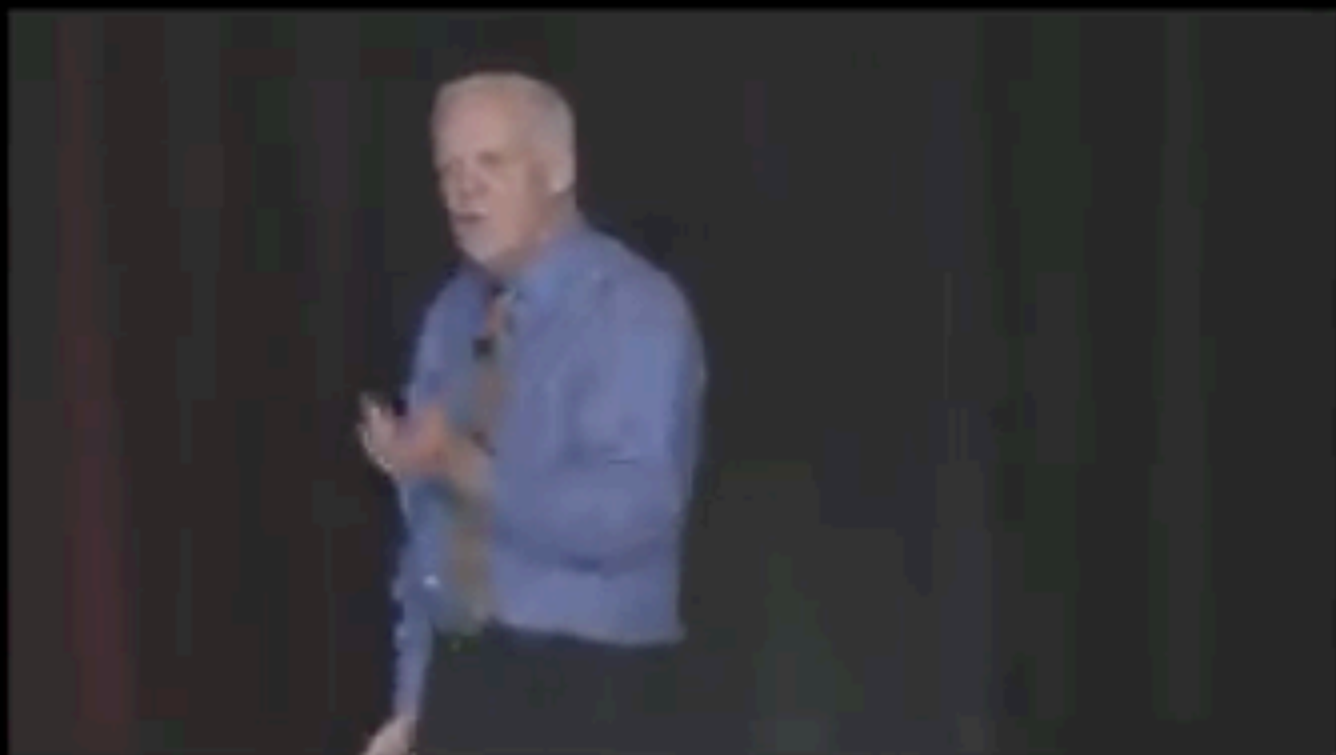
Programming languages

- Which of the following programming language needs to highest instruction count to print "Hello, world!" on screen?
 - A. C
 - B. C++
 - C. Java
 - D. Rust
 - E. Python



Use “performance counters” to figure out!

- Modern processors provides performance counters
 - instruction counts
 - cache accesses/misses
 - branch instructions/mis-predictions
- How to get their values?
 - You may use “perf stat” in linux
 - You may use Instruments —> Time Profiler on a Mac
 - Intel's vtune — only works on Windows w/ intel processors
 - You can also create your own functions to obtain counter values



What's the Opportunity?

Matrix Multiply: relative speedup to a Python version (18 core Intel)

Version	Speed-up	Optimization
Python	1	
C	47	Translate to static, compiled language
C with parallel loops	366	Extract parallelism
C with loops & memory optimization	6,727	Organize parallelism and memory access
Intel AVX instructions	62,806	Use domain-specific HW

from: "There's Plenty of Room at the Top," Leiserson, et. al., *to appear*.

Programming languages & performance

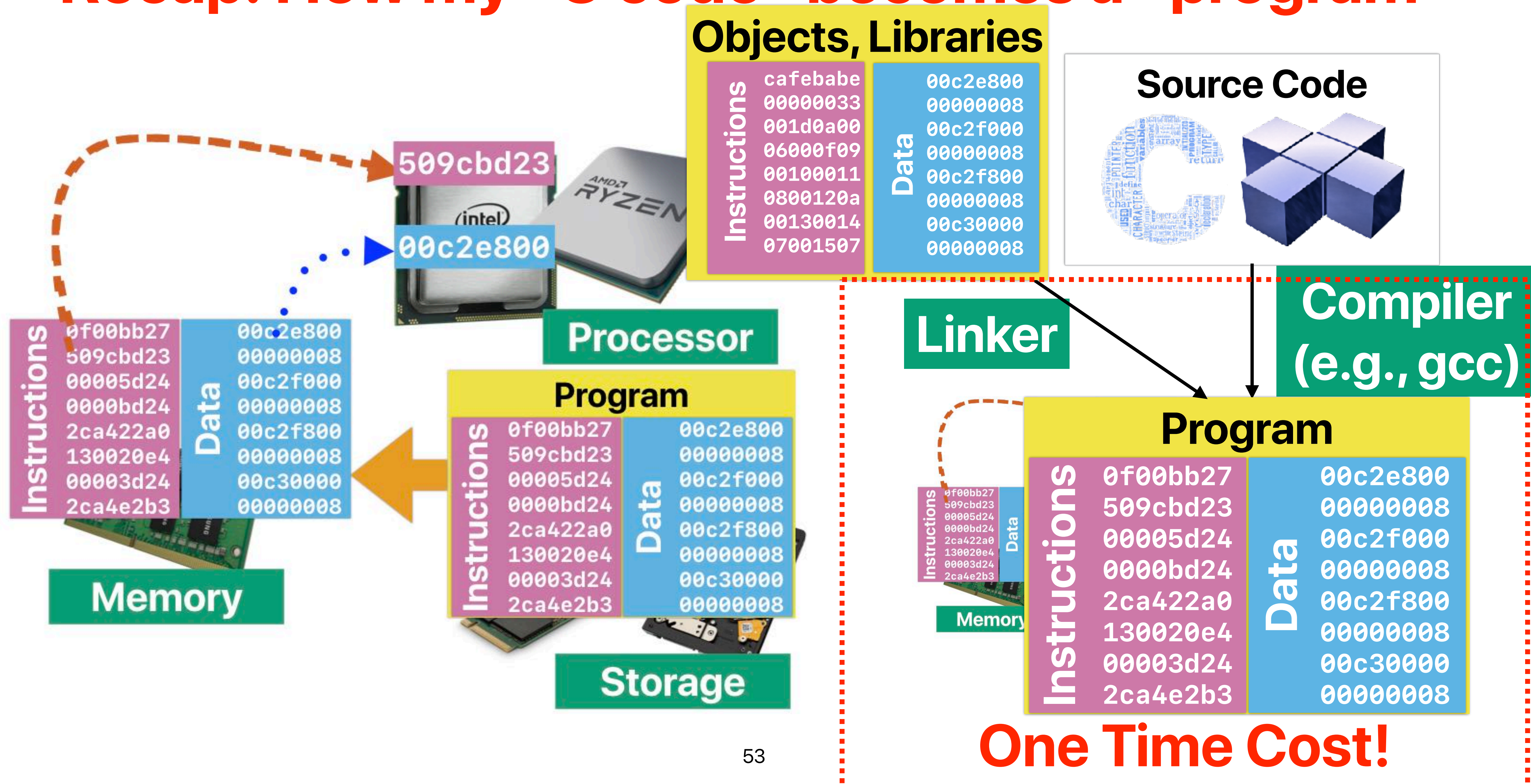
- How many instructions are there in "Hello, world!"

	Instruction count	LOC	Ranking	CPI
C	~1M	6	1	1.04
C++	~3.7M	6	3	0.76
Java	~154M	8	5	0.66
Python	~52M	1	4	0.47
GO (Interpreter)	~1400M	1	6	0.59
GO (Compiled)	~3M	1		1.33
Rust	~1.6M	1	2	1.15

Programming languages

- Which of the following programming language needs to highest instruction count to print "Hello, world!" on screen?
 - A. C
 - B. C++
 - C. Java
 - D. Rust
 - E. Python

Recap: How my "C code" becomes a "program"



Start with this simple program in C

```
int A[] =  
{1,2,3,4,5,6,7,8,9,10,1,2,3,4  
,5,6,7,8,9,10};
```

Compiler

Contents of section .data:

0000	01000000	02000000	03000000	04000000
0010	05000000	06000000	07000000	08000000
0020	09000000	0a000000	0b000000	0c000000
0030	0d000000	0e000000	0f000000	10000000
0040	11000000	12000000	13000000	14000000

control flow

operations

logical operations

```
int main()  
{  
    int i=0, sum=0;  
    for(i = 0; i < 20; i++)  
    {  
        sum += A[i];  
    }  
    return 0;  
}
```

memory access

arithmetic operations

main:

.LFB0:

endbr64

pushq %rbp

movq %rsp, %rbp

movl \$0, -8(%rbp)

movl \$0, -4(%rbp)

movl \$0, -8(%rbp)

jmp .L2

.L3:

movl -8(%rbp), %eax

cltq

leaq 0(,%rax,4), %rdx

leaq A(%rip), %rax

movl (%rdx,%rax),

%eax

addl %eax, -4(%rbp)

addl \$1, -8(%rbp)

.L2:

cmpl \$19, -8(%rbp)

jle .L3

movl \$0, %eax

popq %rbp

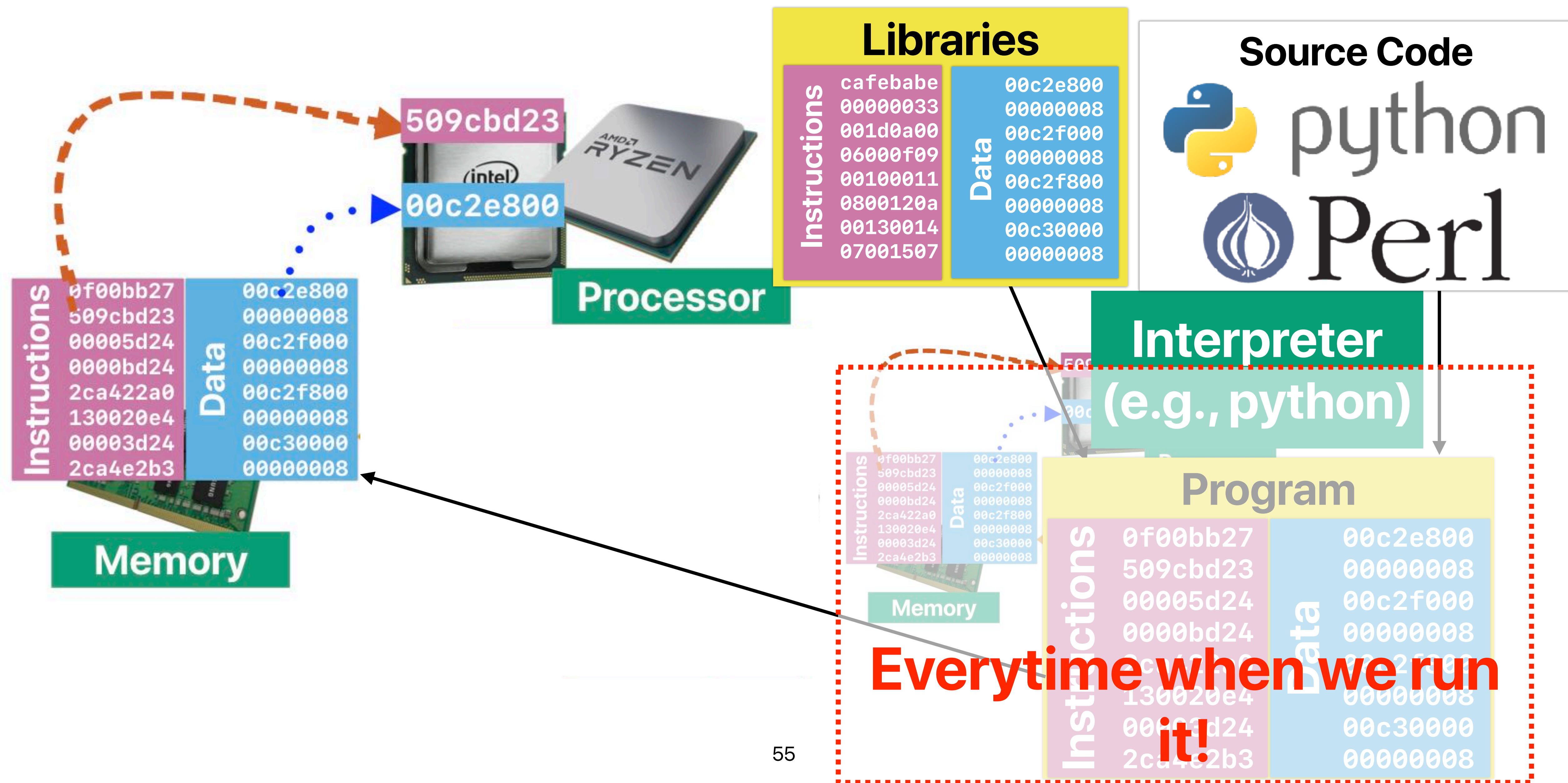
ret

Compiler

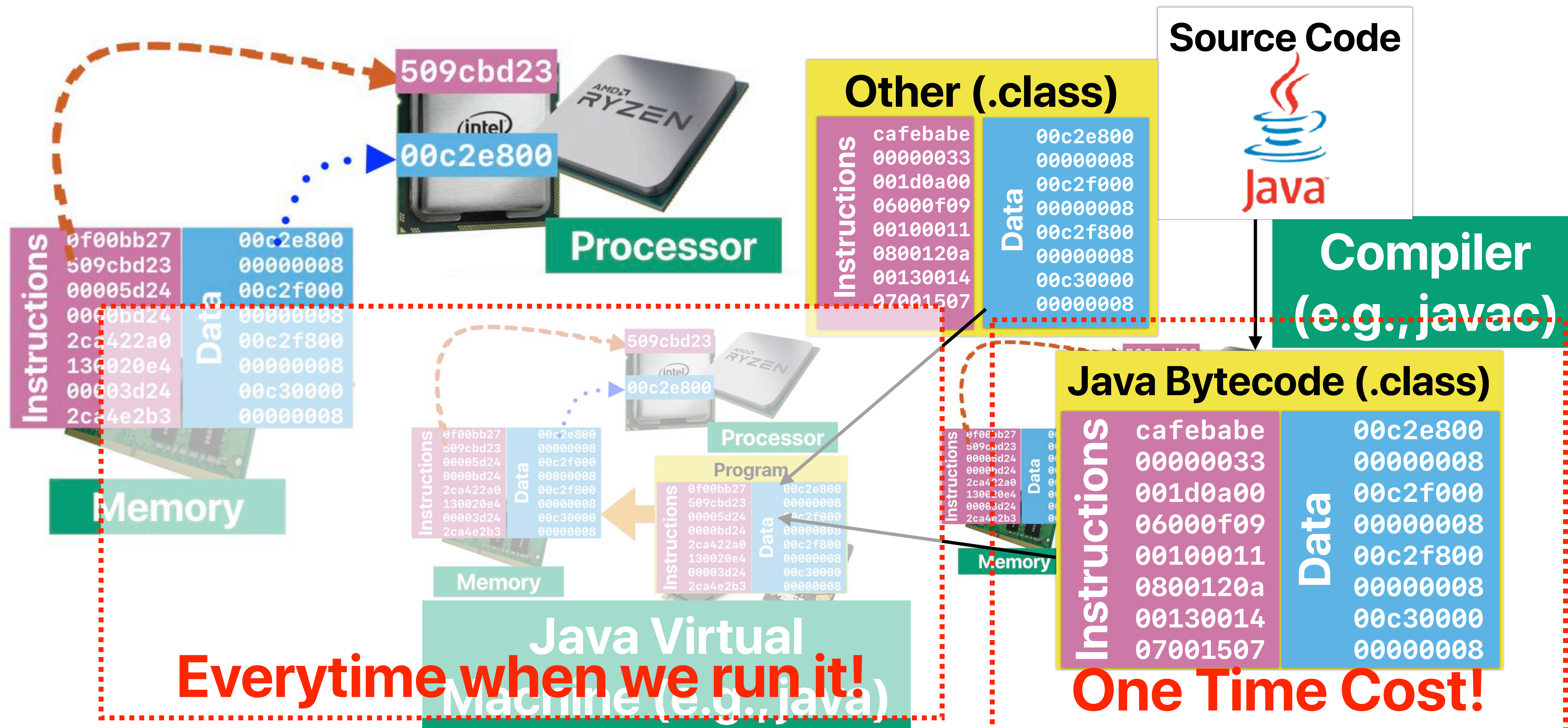
Contents of section .text:

0000	f30f1efa	554889e5	c745f800	000000c7
0010	45fc0000	0000c745	f8000000	00eb1e8b
0020	45f84898	488d1140	50000000	488d0500
0030	0000008b	04020145	fc8345f8	01837df8
0040	137edcb8	00000000	5dc3	

Recap: How my "Python code" becomes a "program"

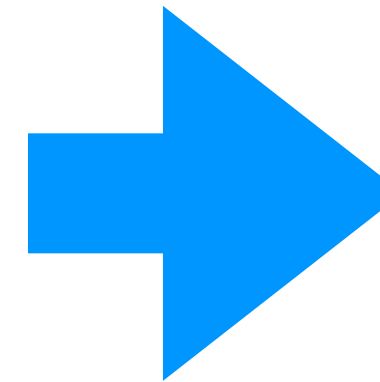


Recap: How my "Java code" becomes a "program"



What's in your java classes?

```
public static int fibonacci(int n) {  
    if(n == 0)  
        return 0;  
    else if(n == 1)  
        return 1;  
    else  
        return fibonacci(n - 1) + fibonacci(n - 2);  
}
```



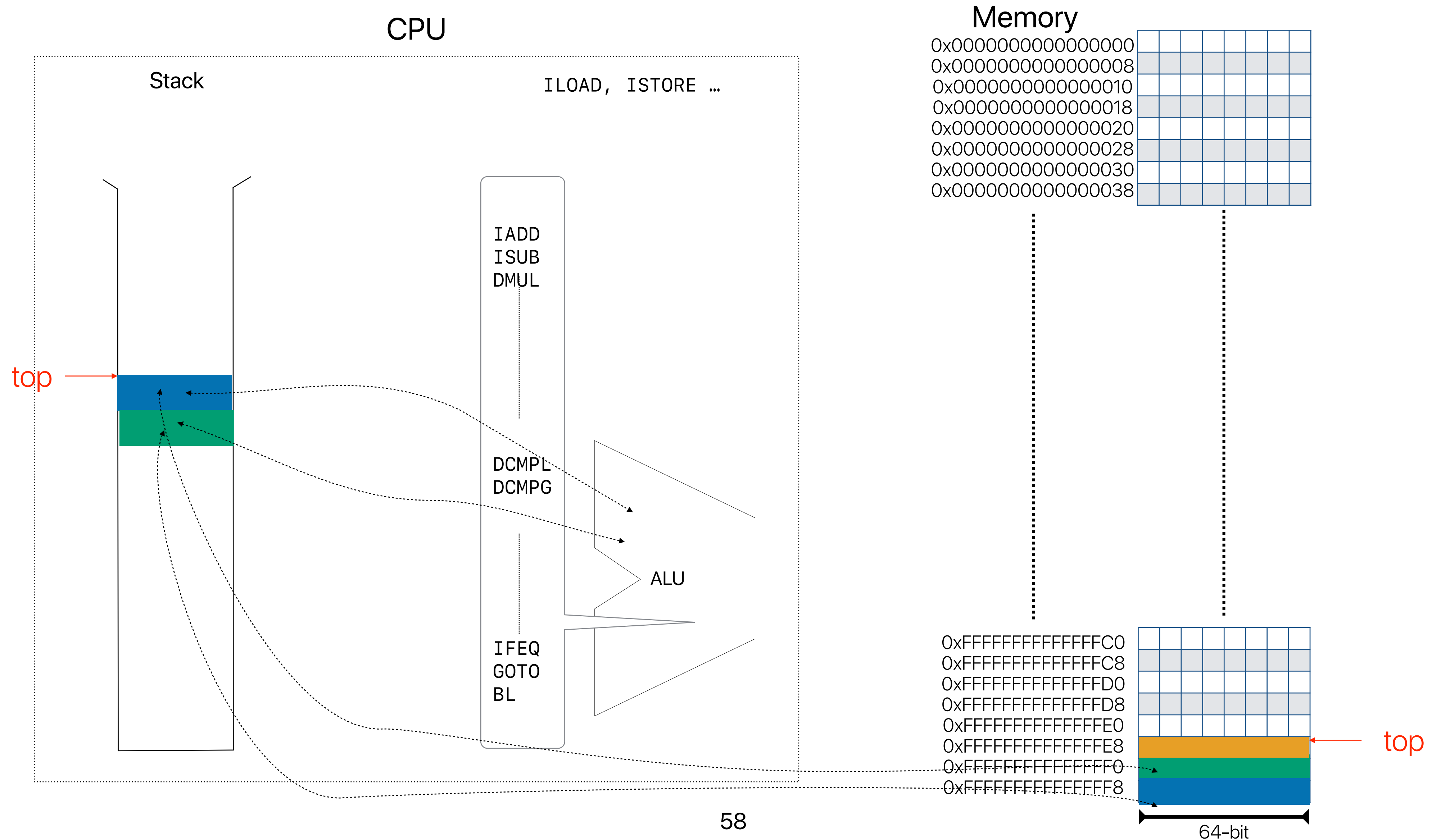
```
0: iload_0  
1: ifne  
4: iconst_0  
5: ireturn  
6: iload_0  
7: iconst_1  
8: if_icmpne  
11: iconst_1  
12: ireturn  
13: iload_0  
14: iconst_1  
15: isub  
16: invokestatic #2  
19: iload_0  
20: iconst_2  
21: isub  
22: invokestatic #2  
25: iadd  
26: ireturn
```



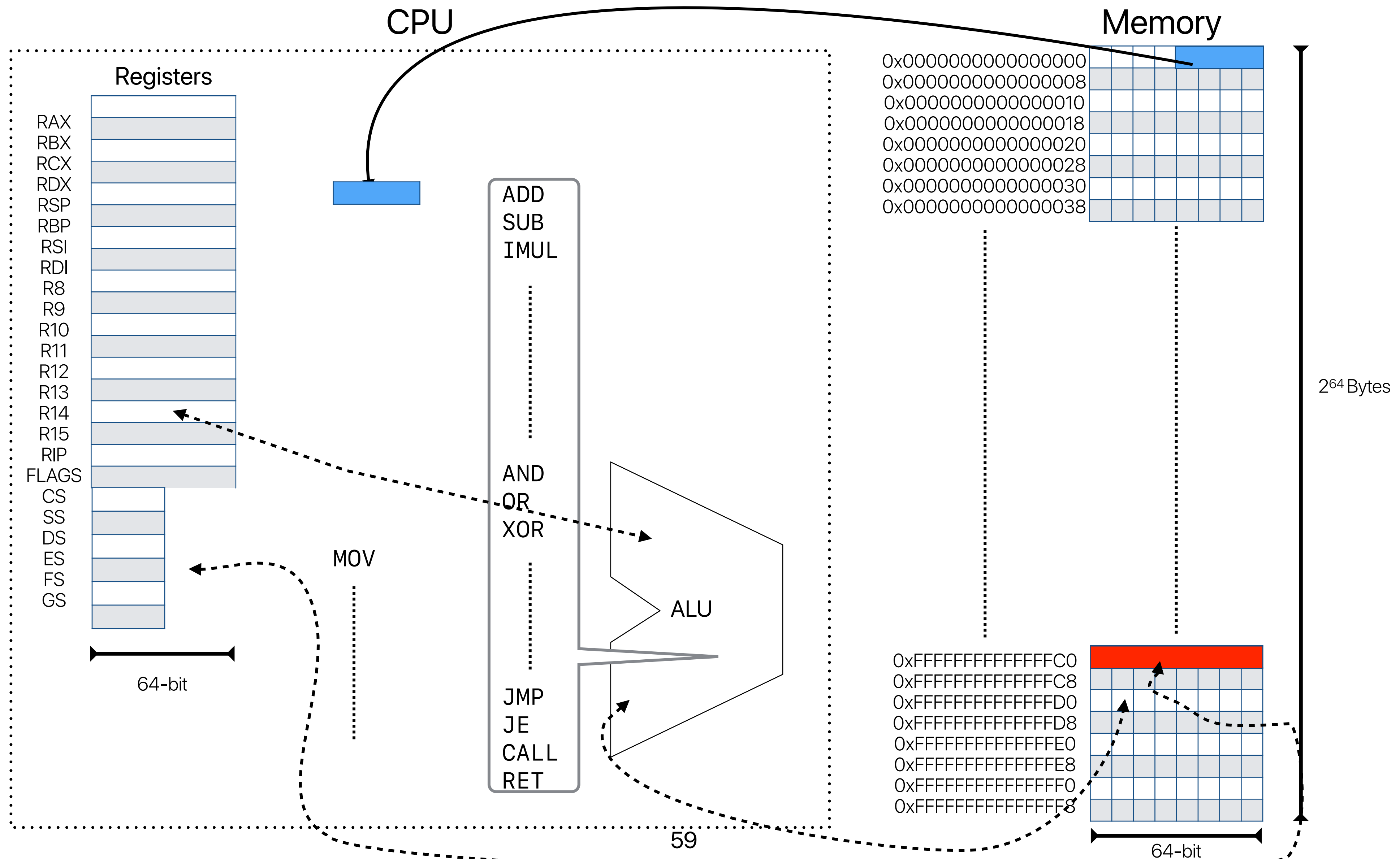
labels

**Most instructions doesn't
have an argument!**

"Abstracted" Java Architecture

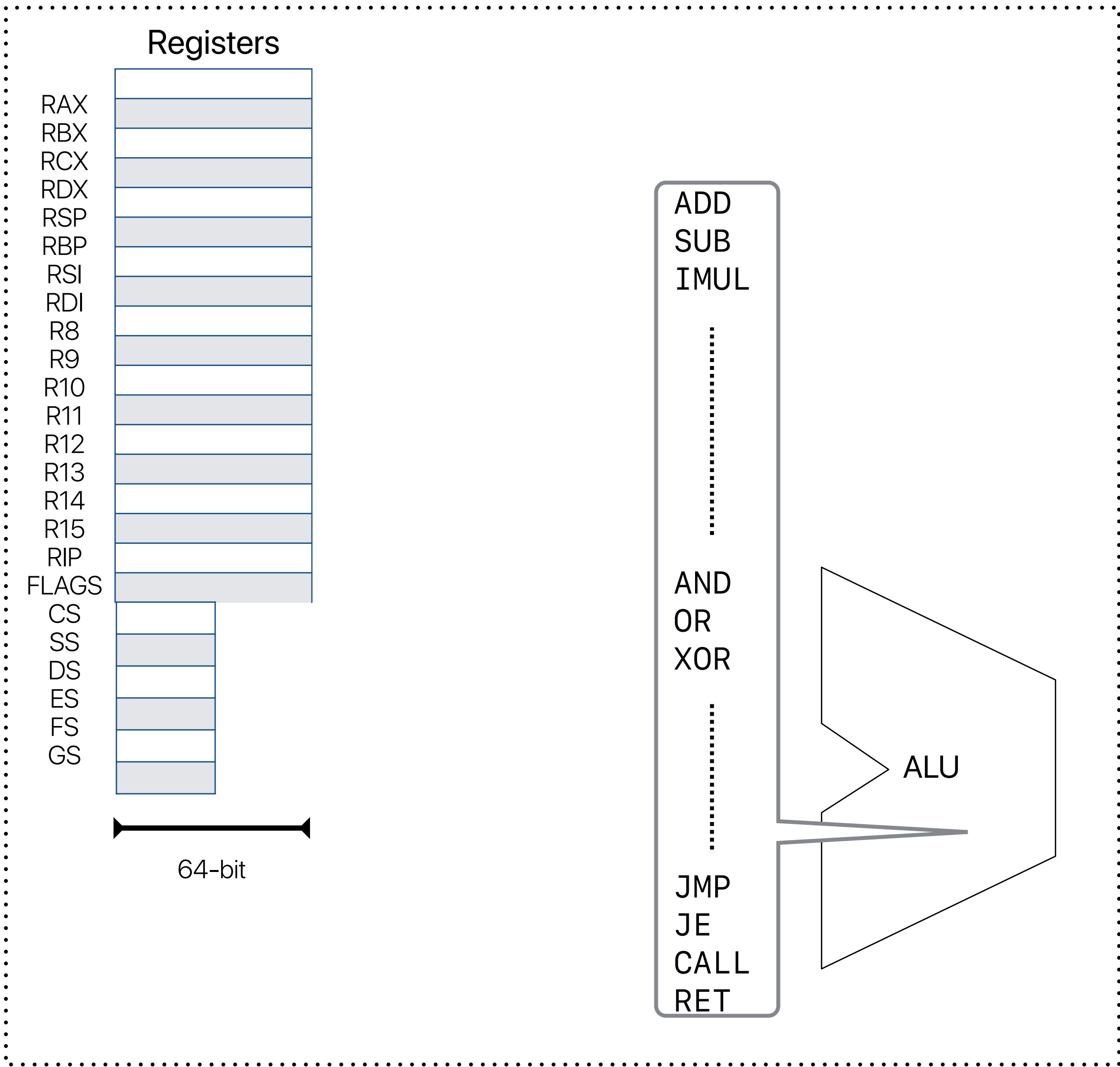


x86 ISA: the abstracted machine

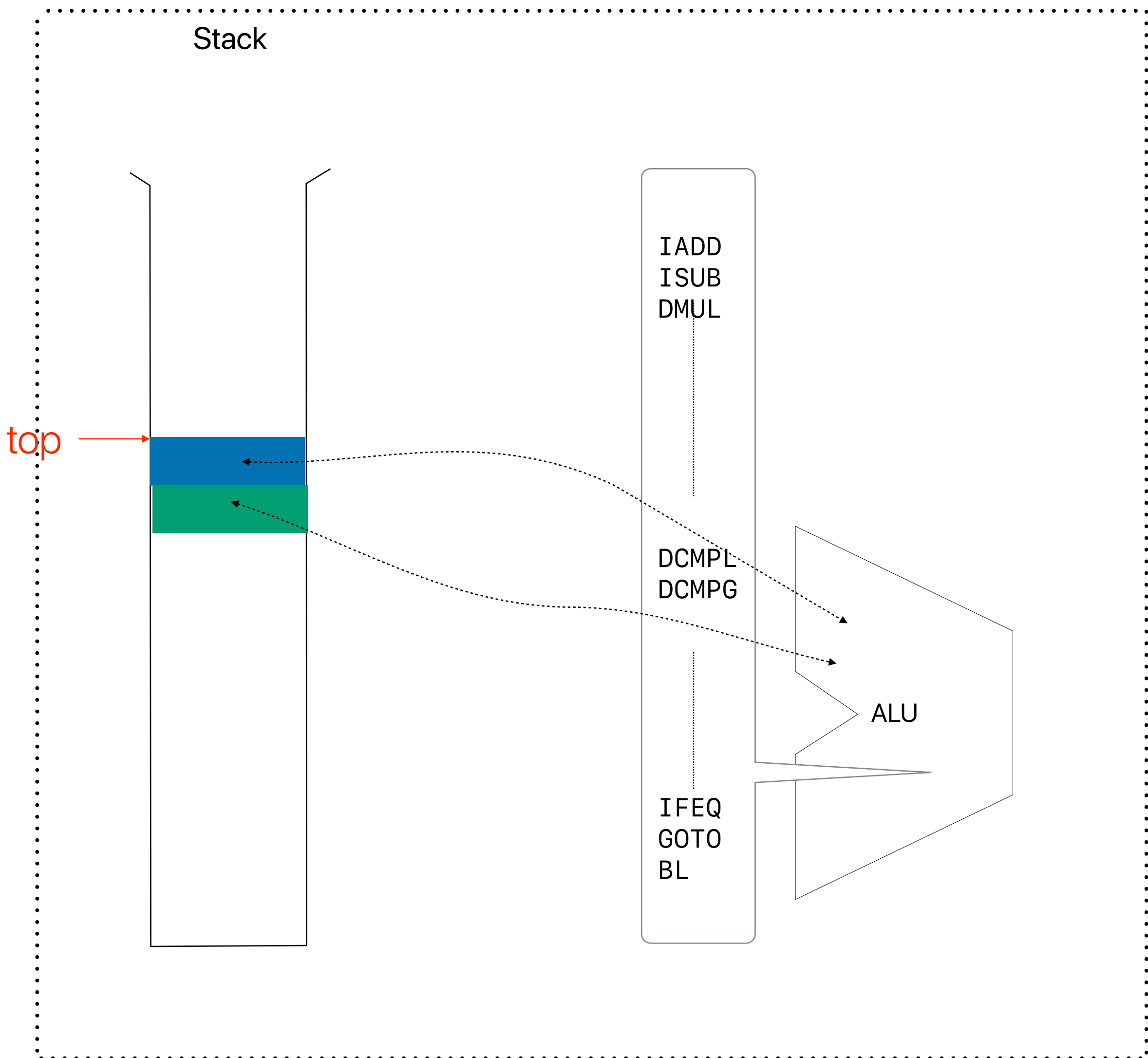


Mismatching real architectures vs JVM abstraction

x86 CPU



JVM's CPU abstraction



Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric — Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions
- Different programming languages can generate machine operations with different orders of magnitude performance — programmers need to make wise choice of that!

**What about programmer &
performance?**

Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

$O(n^2)$

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

$O(n^2)$

Complexity

Instruction Count?

Clock Rate

CPI



Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

How many of the following make(s) the performance different between version A & version B?

- ① IC
- ② CPI
- ③ CT
- A. 0
- B. 1
- C. 2
- D. 3



Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

$O(n^2)$

Complexity

$O(n^2)$

Same

Instruction Count?

Same

Same

Clock Rate

Same

???

CPI

???

Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

$O(n^2)$

Complexity

$O(n^2)$

Same

Instruction Count?

Same

Same

Clock Rate

Same

Better

CPI

Worse

Demo — programmer & performance

A

```
for(i = 0; i < ARRAY_SIZE; i++)  
{  
    for(j = 0; j < ARRAY_SIZE; j++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)  
{  
    for(i = 0; i < ARRAY_SIZE; i++)  
    {  
        c[i][j] = a[i][j]+b[i][j];  
    }  
}
```

How many of the following make(s) the performance different between version A & version B?

- ① IC
- ② ☒ CPI
- ③ CT

A. 0

B. 1

C. 2

D. 3

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric — Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Different programming languages can generate machine operations with different orders of magnitude performance — programmers need to make wise choice of that!
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions
- Programmers can control all three factors in the classic performance equation when composing the program

Compilers & performance



How compilers affect performance

- If we turn on "-O3" in gcc when compiling both code snippets **A** and **B**, how many of the following can we expect?

- ① Compiler optimizations can reduce IC for both
- ② Compiler optimizations can make the CPI lower for both
- ③ Compiler optimizations can make the ET lower for both
- ④ Compiler optimizations can transform code B into code A

A. 0

B. 1

C. 2

D. 3

E. 4

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

How compilers affect performance

- If we turn on "-O3" in gcc when compiling both code snippets **A** and **B**, how many of the following can we expect?

①  Compiler optimizations can reduce IC for both
Compiler can apply loop unrolling, constant propagation naively to reduce IC

② Compiler optimizations can make the CPI lower for both
Reduced IC does not necessarily mean lower CPI — compiler may pick one longer instruction to replace a few shorter ones

③ Compiler optimizations can make the ET lower for both
Compiler cannot guarantee the combined effects lead to better performance!

④ Compiler optimizations can transform code B into code A
Compiler will not significantly change programmer's code since compiler cannot guarantee if doing that would affect the correctness

A. 0

B. 1

C. 2

D. 3

E. 4

A

```
for(i = 0; i < ARRAY_SIZE; i++)
{
    for(j = 0; j < ARRAY_SIZE; j++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

B

```
for(j = 0; j < ARRAY_SIZE; j++)
{
    for(i = 0; i < ARRAY_SIZE; i++)
    {
        c[i][j] = a[i][j]+b[i][j];
    }
}
```

Takeaways: What is and what affects “performance”?

- Latency is the most fundamental performance metric — Instruction count, cycles per instruction, cycle time define the latency of execution on CPUs
- Throughput metrics (e.g., FLOPS) do not consider all aspects in performance equations and therefore can mislead important application/system decisions
- Different programming languages can generate machine operations with different orders of magnitude performance — programmers need to make wise choice of that!
- Programmers can control all three factors in the classic performance equation when composing the program
- Compiler optimization does not always help
 - Compiler optimizes code based on some assumptions that may not be true on all computers
 - Programmers' can write code in a way facilitating optimizations!

Announcement

- Reading quiz due next Tuesday before the lecture
 - We will drop two of your least performing reading quizzes
 - Once you closed, you cannot restart
- Assignment 0 **due this Thursday**
 - We cannot help you at the last minute — please start early
 - Watch before you start https://youtu.be/m7OoY8y_Isk
 - Please always make sure you follow the exact steps in the readme and the notebook
 - Submit to the right item on Gradescope — Gradescope can give you feedback and you're allowed to resubmit
 - Please visit an office hour if you need more assistance
 - Hung-Wei Tseng
 - Tu 9p-10p @ Google Meet
 - W 1p-2p @ WCH 406
- Check our website for slides/quiz links, Gradescope to submit assignments, piazza for discussions
- Check your grades at https://www.escalab.org/my_grades
 - If you don't have any grade, you need to make sure your gradescope account is associated with your UCRNetID@ucr.edu
 - You have to submit a course agreement to receive scores
- Youtube channel for lecture recordings:
<https://www.youtube.com/c/ProfUsagi/playlists>

Computer Science & Engineering

203

つづく

